UNIVERSIDAD ADVENTISTA DE BOLIVIA

Ingeniería de Sistemas

Tecnologías de Internet

Proyecto Integrador — Segunda Entrega

Sistema Académico Escolar para Gestión de Colegios

Implementación Full-Stack: React · Node.js/Express · MongoDB

Integrantes

1. Juan Daniel Ancieta Toledo
2. Jazmin Cielo Canaviri Mamani
3. Diego Alejandro Mendez Benitez
4. Saul Lander Reyes Guzman

Docente

Victor Hugo Sarzuri Flores

Repositorio GitHub

https://github.com/<usuario>/sistema-academico-escolar

Fecha

10/11/2026

COCHABAMBA-BOLIVIA

ÍNDICE

1. Introducción y alcance
2. Arquitectura del sistema
3. Modelo de datos
4. Backend: seguridad, autenticación y autorización
5. API REST y documentación Swagger
6. Frontend
7. Trazabilidad con la primera entrega (RF y RNF)
8. Pruebas realizadas
9. Instalación, ejecución y despliegue
10. Decisiones, limitaciones y trabajo futuro
11. Conclusiones
Anexo A. Catálogo completo de endpoints

1. Introducción y alcance

La primera entrega definió el **Sistema Académico Escolar**: requisitos funcionales y no funcionales, casos de uso, diagrama C4 de contexto, investigación UX y un prototipo navegable en Figma. Esta segunda entrega convierte ese diseño en un producto **funcional y desplegable**: una aplicación web full-stack con API REST, base de datos MongoDB, autenticación con JWT, control de acceso por roles y una interfaz React que consume la API.

El sistema cubre los cuatro roles del diseño original (Administrador, Docente, Estudiante y Apoderado/Tutor) y los diez requisitos funcionales RF-001 a RF-010. El código fuente completo, con historial de commits y README de instalación, está en el repositorio indicado en la carátula.

1.1 Stack tecnológico

Capa	Tecnología	Justificación
Frontend	React 18, Vite, Tailwind CSS 4, React Router 6, Axios	Componentes reutilizables, build rápido, diseño responsive con utilidades y paleta institucional (#084298) del Figma.
Backend	Node.js ≥ 18, Express 4	API REST ligera; mismo lenguaje (JavaScript) en todo el proyecto.
Base de datos	MongoDB + Mongoose 8	Documentos flexibles, esquemas con validación, índices únicos y compuestos.
Seguridad	jsonwebtoken, bcryptjs, helmet, cors, express-rate-limit, multer	JWT con expiración, hash con salt ≥ 10, cabeceras seguras, límite de intentos y subida de archivos restringida.
Documentación	swagger-ui-express (OpenAPI 3.0)	Documentación interactiva en `/api-docs` con autenticación Bearer.
Pruebas	node:test + supertest, Playwright	13 pruebas unitarias, 9 de integración con MongoDB real y verificación de la interfaz en navegador.

1.2 Módulos implementados

- **Autenticación y usuarios:** registro, login con contraseña, login con código OTP por correo, perfil y gestión de usuarios (CRUD) por el administrador.
- **Gestión académica:** materias con docente y estudiantes matriculados, carga y publicación de calificaciones, boletín en PDF, toma de asistencia diaria.
- **Tareas:** creación y asignación por el docente, entrega de archivos por el estudiante, revisión de entregas.
- **Solicitudes:** justificación de inasistencias por el apoderado con comprobante y aprobación por Regencia/administrador.
- **Notificaciones y panel:** alertas internas y por correo; dashboard con métricas adaptadas al rol.

2. Arquitectura del sistema

El sistema sigue una arquitectura cliente-servidor en tres capas: una **SPA** en React que presenta la interfaz, una **API REST** en Express que concentra la lógica de negocio y la seguridad, y **MongoDB** como persistencia. La SPA nunca accede a la base de datos: toda petición pasa por la API, que valida el token, verifica el rol y filtra los datos por pertenencia.

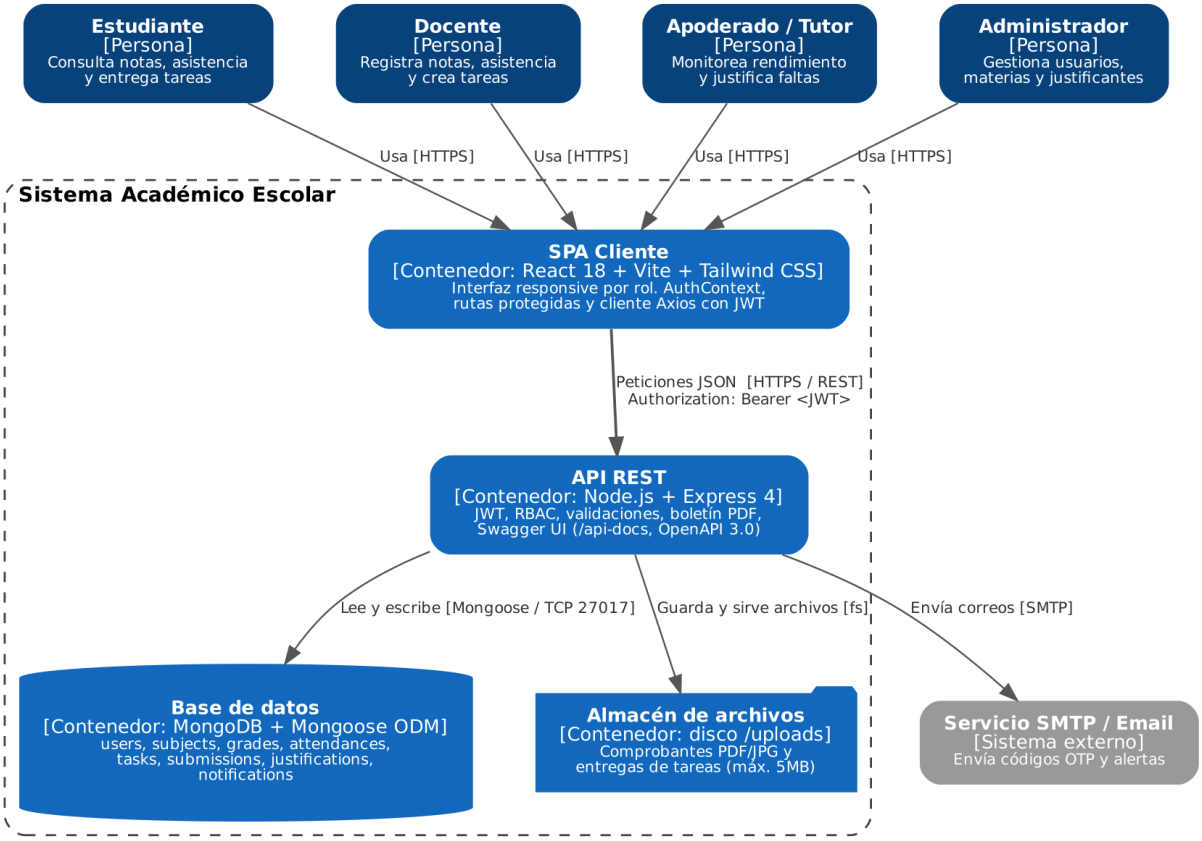


Figura 1. Diagrama C4 – Nivel 2 (Contenedores). Refina el diagrama de contexto de la primera entrega.

2.1 Estructura por capas del backend

Carpeta	Responsabilidad
`config/`	Lectura y validación de variables de entorno (el servidor no arranca sin `JWT_SECRET`) y conexión a MongoDB.
`models/`	Esquemas Mongoose: tipos, obligatoriedad, rangos, enumeraciones, índices únicos y compuestos.
`middlewares/`	`authenticate` (verifica JWT y carga el usuario), `authorize(...roles)` (RBAC), `upload` (multer con filtro de tipo y 5 MB), `errorHandler` (respuestas JSON uniformes).
`routes/`	Mapa de endpoints y rol requerido por cada uno, en un único archivo para auditar permisos de un vistazo.
`controllers/`	Lógica de negocio por módulo: auth, users, subjects, grades, attendance, tasks, justifications, notifications, dashboard, reports.
`services/`	Reglas compartidas: alcance de datos por rol (`access.js`), boletín y asistencia (`academic.js`), notificaciones y correo (`notify.js`, `mailer.js`).
`docs/`	Especificación OpenAPI 3.0 servida con Swagger UI.

Decisión de diseño — autorización por pertenencia. No basta con comprobar el rol: un docente solo opera sobre **sus** materias, un estudiante solo ve **sus** datos y un apoderado solo los de **sus hijos**. Esta

regla vive en `services/access.js` y se aplica en cada controlador, de modo que un usuario con un token válido no puede leer datos ajenos cambiando un identificador en la URL (previene IDOR).

3. Modelo de datos

Se definieron **8 colecciones** relacionadas por referencias (`ObjectId`). Cada esquema valida sus datos antes de persistirlos y declara los índices que respaldan las consultas más frecuentes.

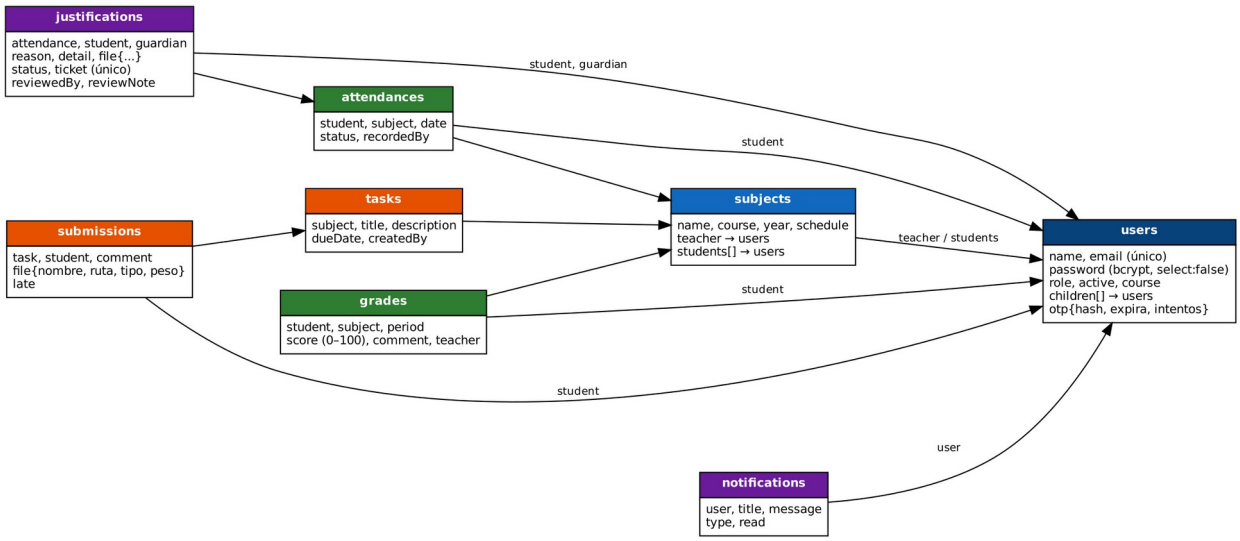


Figura 2. Colecciones MongoDB y sus referencias.

Colección	Campos clave y validaciones	Índices
`users`	name (3–100), email (formato válido, único, minúsculas), password (hash bcrypt, `select:false`), role ∈ {administrador, docente, estudiante, apoderado}, active, children[] (apoderado), otp.	email único; (role, active); texto en name
`subjects`	name, course, year, schedule, teacher (obligatorio, rol docente), students[] (rol estudiante).	único (name, course, year); teacher; students
`grades`	student, subject, period ∈ {T1,T2,T3}, score 0–100 (min/max), comment, teacher.	único (student, subject, period); (subject, period)
`attendances`	student, subject, date (día UTC), status ∈ {presente, ausente, tardanza, justificada}, recordedBy.	único (student, subject, date); (subject, date); (student, status)
`tasks`	subject, title (3–120), description, dueDate (obligatoria), createdBy.	(subject, dueDate)
`submissions`	task, student, comment, file{nombre, ruta, tipo, peso}, late.	único (task, student)
`justifications`	attendance, student, guardian, reason ∈ {Salud, Familiar, Fuerza Mayor}, detail, file, status ∈ {Pendiente, Aprobada, Rechazada}, ticket único, reviewedBy.	único parcial por attendance (solo Pendiente/Aprobada); (status, createdAt); guardian
`notifications`	user, title, message, type, read.	(user, read, createdAt)

Los **índices únicos** son una segunda barrera de integridad, independiente del código: aunque dos peticiones simultáneas intenten guardar la misma nota o justificar la misma falta, la base de datos rechaza el duplicado (código 11000 → HTTP 409). El índice único *parcial* de `justifications` implementa el flujo alternativo A2 del UC-08 (una falta con solicitud pendiente o aprobada no admite otra) y a la vez permite volver a solicitar si fue rechazada.

4. Backend: seguridad, autenticación y autorización

4.1 Autenticación con JWT

El inicio de sesión (`POST /api/auth/login`) busca al usuario por correo, compara la contraseña con **bcrypt** y, si es correcta, firma un **JWT** con `sub` (id) y `role`, vigencia configurable (por defecto 1 hora). Credenciales erróneas y usuario inexistente producen el mismo mensaje (“Credenciales inválidas”) para no revelar qué cuentas existen. La contraseña se guarda siempre con hash (salt ≥ 10) y el campo nunca sale en las respuestas ni en las consultas por defecto (`select:false` + `toJSON`).

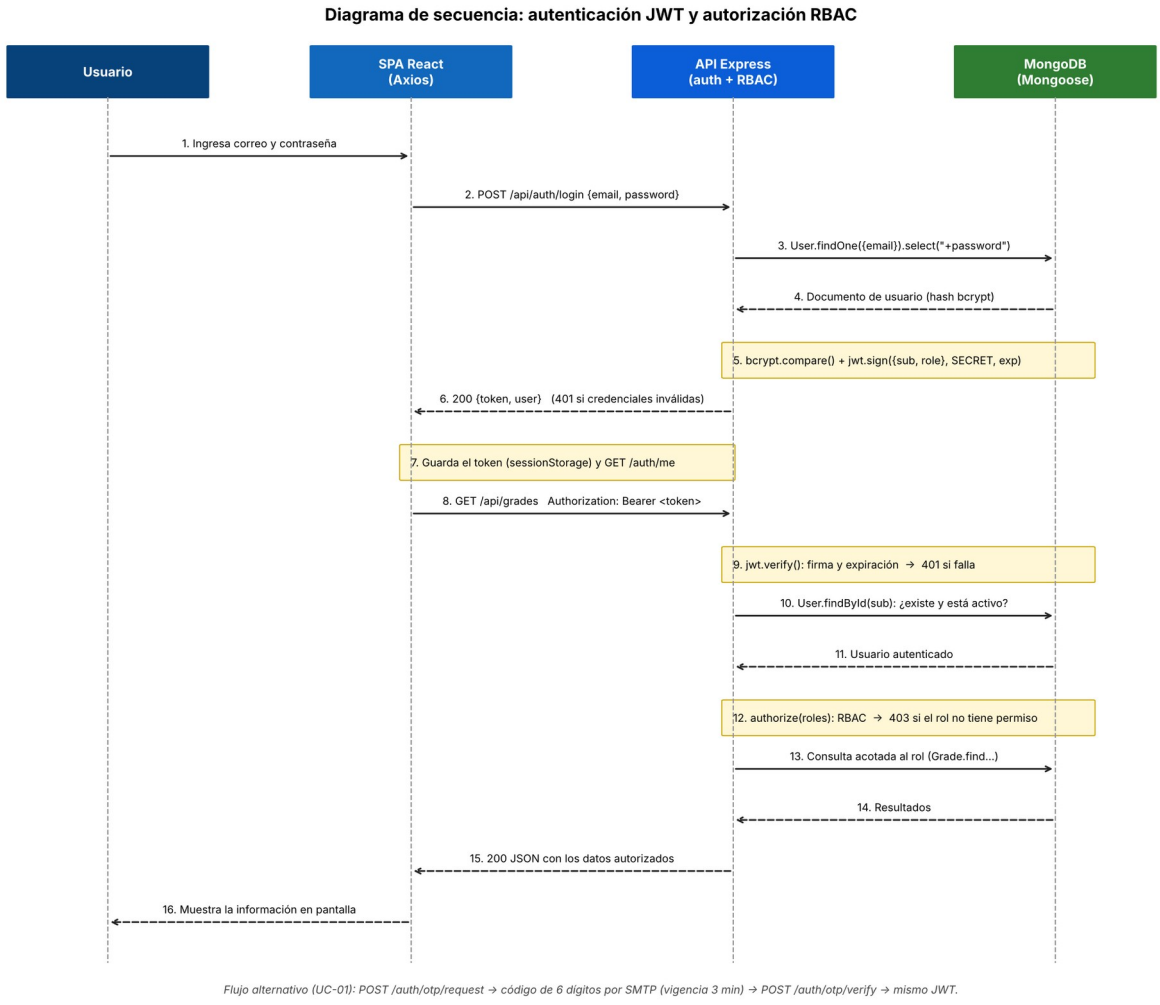


Figura 3. Secuencia de autenticación JWT y autorización RBAC.

4.2 Código OTP por correo (UC-01)

Se conservó el flujo de la primera entrega: `POST /auth/otp/request` genera un código de 6 dígitos con `crypto.randomInt`, guarda **solo su hash SHA-256** con vigencia de 3 minutos y lo envía por SMTP. `POST /auth/otp/verify` lo compara en tiempo constante (`timingSafeEqual`), limita los intentos a 5 y lo invalida tras el primer uso. Los mensajes “Código inválido o expirado” y “Usuario no encontrado en la base de datos” corresponden a los flujos alternativos A1 y A2 del caso de uso. Sin SMTP configurado, el correo se imprime en consola, lo que permite probar todo localmente.

4.3 Autorización por roles (RBAC)

El middleware `authenticate` exige `Authorization: Bearer <token>`, verifica firma y expiración, recarga al usuario desde la base de datos y rechaza cuentas desactivadas (401). El middleware `authorize(...roles)` responde 403 si el rol no está permitido. La matriz completa está en el README; en resumen:

Recurso	Administrador	Docente	Estudiante	Apoderado
Usuarios y materias (CRUD)	Sí	No	No	No
Cargar notas / asistencia / tareas	Sí	Solo sus materias	No	No
Ver boletín y asistencia	Todos	Sus materias	Los propios	Los de sus hijos
Entregar tareas	No	No	Sí	No
Solicitar justificación	No	No	No	Sí
Aprobar / rechazar justificación	Sí	No	No	No

4.4 Otras medidas de seguridad

- **Validación de entrada** en dos niveles: validaciones del esquema Mongoose y comprobaciones explícitas en controladores (formato de fecha, rango 0–100, ids válidos, estudiantes realmente matriculados). Los errores se devuelven como JSON con código 400 y detalle por campo.
- **Subida de archivos:** solo tipos permitidos (PDF/JPG para comprobantes), máximo 5 MB, nombre aleatorio en disco (evita colisiones y *path traversal*) y descarga solo para usuarios autorizados.
- **Registro público restringido:** solo permite los roles estudiante y apoderado; docentes y administradores los crea el administrador (evita escalada de privilegios).
- **Cabeceras y abuso:** `helmet`, CORS limitado al origen del frontend, límite de peticiones en rutas de autenticación y cuerpo JSON máximo de 1 MB.
- **Secretos fuera del código:** `.env` ignorado por Git; se entrega `.env.example`. En producción el servidor exige un `JWT_SECRET` de al menos 32 caracteres.
- **Baja lógica de usuarios:** se desactivan en lugar de borrarse para conservar el historial académico; un administrador no puede desactivarse a sí mismo.
- **Manejo centralizado de errores:** respuestas uniformes (`{ message, details }`), sin filtrar trazas en producción; traduce errores de Mongoose, JWT y multer a códigos HTTP correctos (400, 401, 403, 404, 409).

5. API REST y documentación Swagger

La API expone **42 endpoints** bajo el prefijo `/api`, con verbos HTTP semánticos (GET, POST, PUT, PATCH, DELETE) y códigos de estado coherentes: 200/201/204 en éxito, 400 por datos inválidos, 401 sin sesión válida, 403 sin permiso, 404 recurso inexistente y 409 conflicto (duplicados). Las listas admiten filtros y paginación (`page`, `limit`, máx. 100).

La documentación **OpenAPI 3.0** se sirve con Swagger UI en `http://localhost:5000/api-docs` (JSON en `/api-docs.json` y en `docs/openapi.json`). Cada operación indica su descripción, parámetros, cuerpo de ejemplo, respuestas posibles y los **roles permitidos**. Para probar rutas protegidas: ejecutar `POST /api/auth/login`, pulsar **Authorize** y pegar el token.

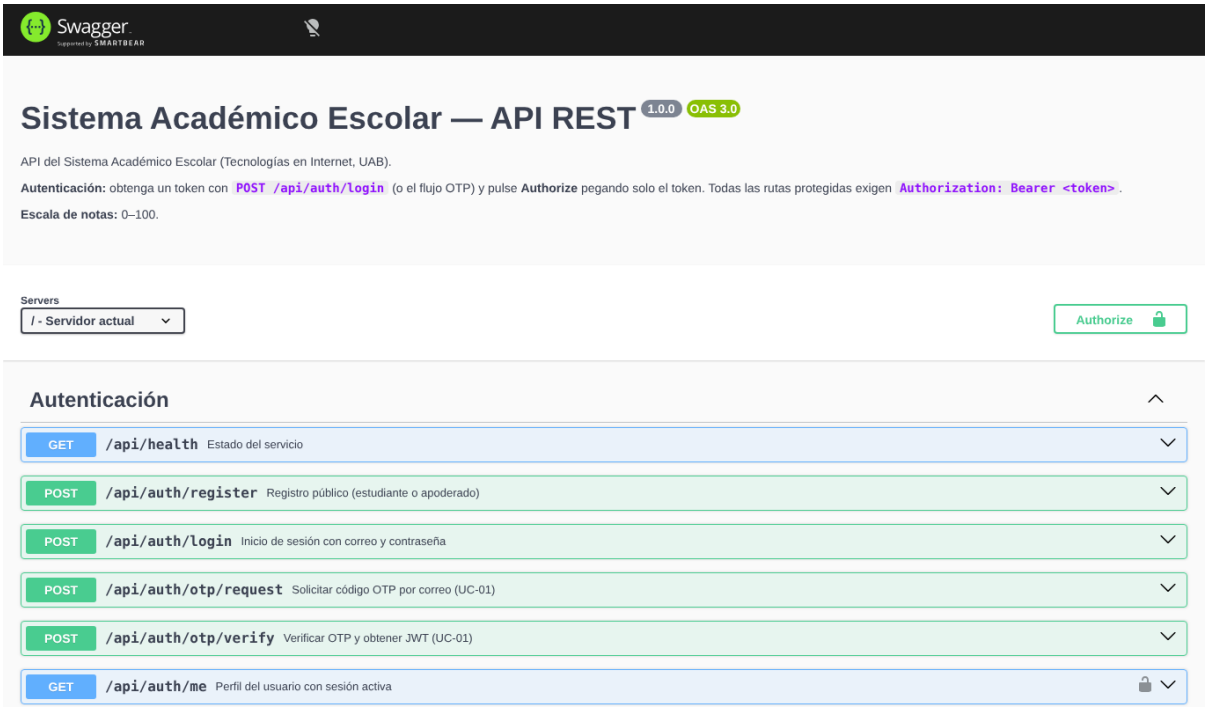


Figura 4. Swagger UI en /api-docs con los endpoints agrupados por módulo.

El catálogo completo de endpoints, con método, ruta, roles y códigos de respuesta, se encuentra en el **Anexo A**.

6. Frontend

La SPA reproduce la identidad visual del prototipo de Figma: azul institucional #084298 y un color de barra lateral por rol (azul para docente y estudiante, verde para apoderado, morado para administrador). El estado de sesión se gestiona con un `AuthContext`; el token se guarda en `sessionStorage` y un interceptor de Axios lo adjunta a cada petición. Ante un 401 (token vencido) la aplicación cierra la sesión y redirige al login.

6.1 Rutas y vistas

Ruta	Vista	Roles
`/login`, `/registro`	Login con OTP o contraseña; registro público	Público
`/`	Dashboard con métricas según el rol (RF-003)	Todos
`/usuarios`	Tabla con búsqueda, filtro por rol, crear, editar y activar/desactivar	Administrador
`/materias`	CRUD de materias: docente titular y matrícula	Administrador
`/calificaciones`	Docente: planilla con validación 0–100 y “Guardar y Publicar”. Estudiante/Apoderado: boletín y descarga PDF	Docente, Estudiante, Apoderado
`/asistencia`	Docente: lista del día. Estudiante/Apoderado: récord, porcentaje y botón “Justificar”	Docente, Estudiante, Apoderado
`/tareas`	Docente: crear/editar/ver entregas. Estudiante: entregar archivo. Apoderado: consulta	Docente, Estudiante, Apoderado
`/justificaciones`	Apoderado: seguimiento por ticket. Administrador: aprobar o rechazar	Apoderado, Administrador
`/notificaciones`, `/perfil`	Centro de notificaciones y datos de la cuenta	Todos

6.2 Experiencia de usuario

- **Estados de carga, vacío y error** en todas las vistas (indicador de carga, mensajes “sin registros” y alertas con el mensaje que devuelve la API, incluido el caso de servidor inaccesible).
- **Validación en cliente y servidor:** p. ej. la celda de nota fuera de rango se resalta en rojo y se inhabilita el guardado (UC-03, A1); el comprobante se limita a PDF/JPG de 5 MB (UC-08, A1).
- **Rutas protegidas por rol** en el cliente (`Protected`) como ayuda de navegación; la seguridad real está en la API.
- **Diseño responsive:** barra lateral colapsable con menú hamburguesa en pantallas pequeñas; tablas con desplazamiento horizontal.
- **Accesibilidad:** etiquetas `label` asociadas, roles ARIA en pestañas y diálogos, cierre con Escape, enlace “Saltar al contenido”, foco visible y no depender solo del color (los estados llevan texto).

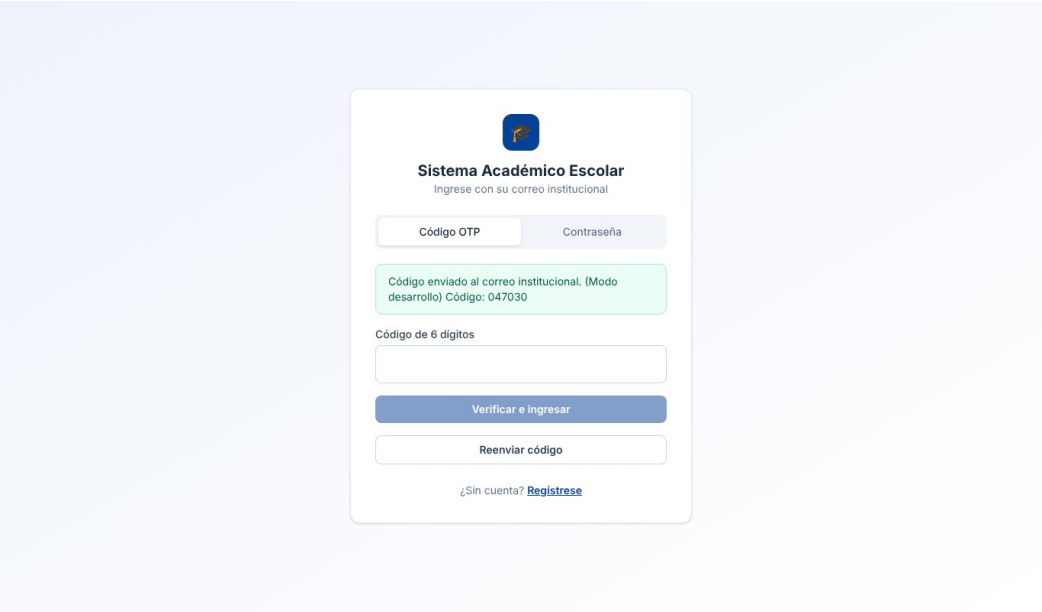


Figura 5. Login con verificación OTP (modo desarrollo muestra el código).



Figura 6. Docente: planilla de calificaciones (UC-03).

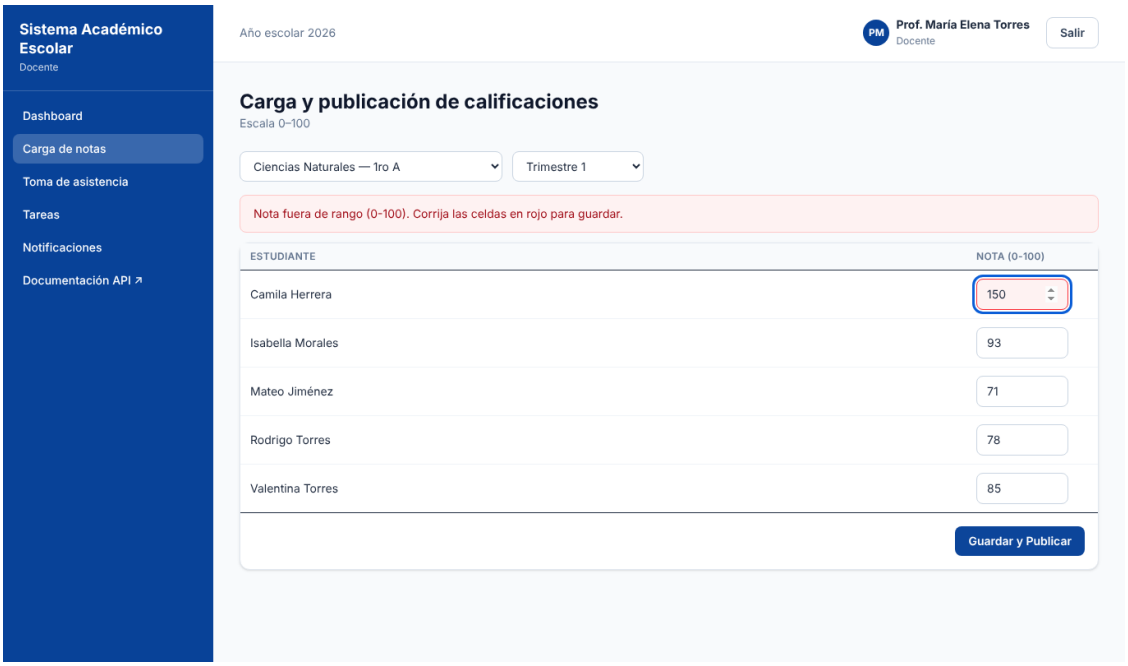


Figura 7. Validación “Nota fuera de rango (0-100)” con guardado inhabilitado.

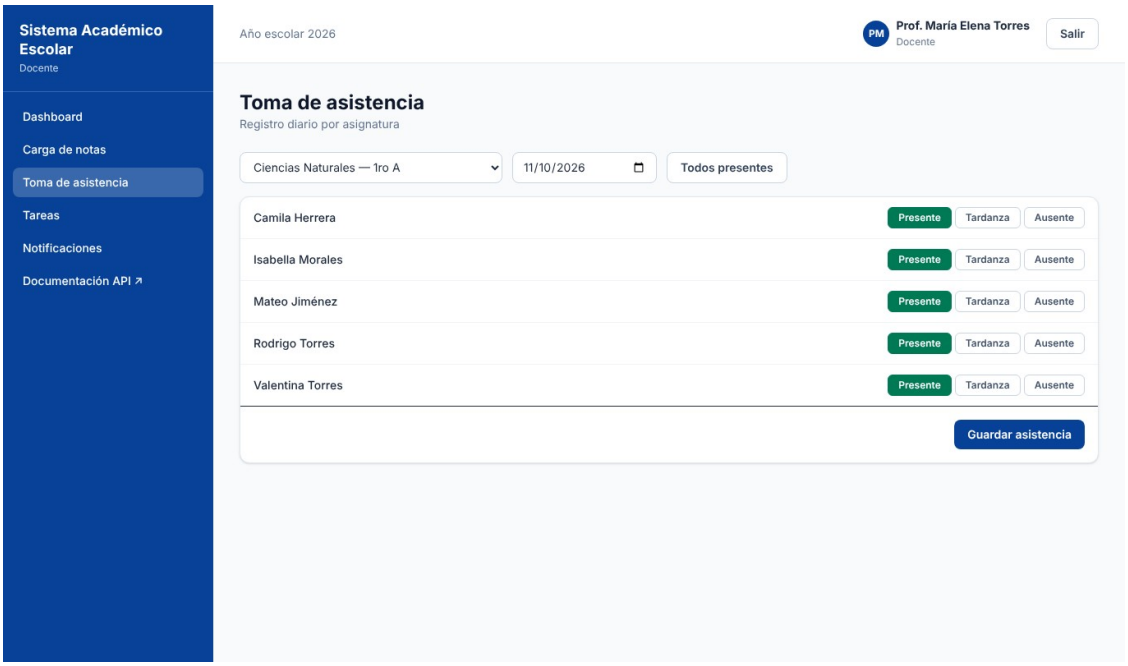


Figura 8. Docente: toma de asistencia diaria (RF-005).

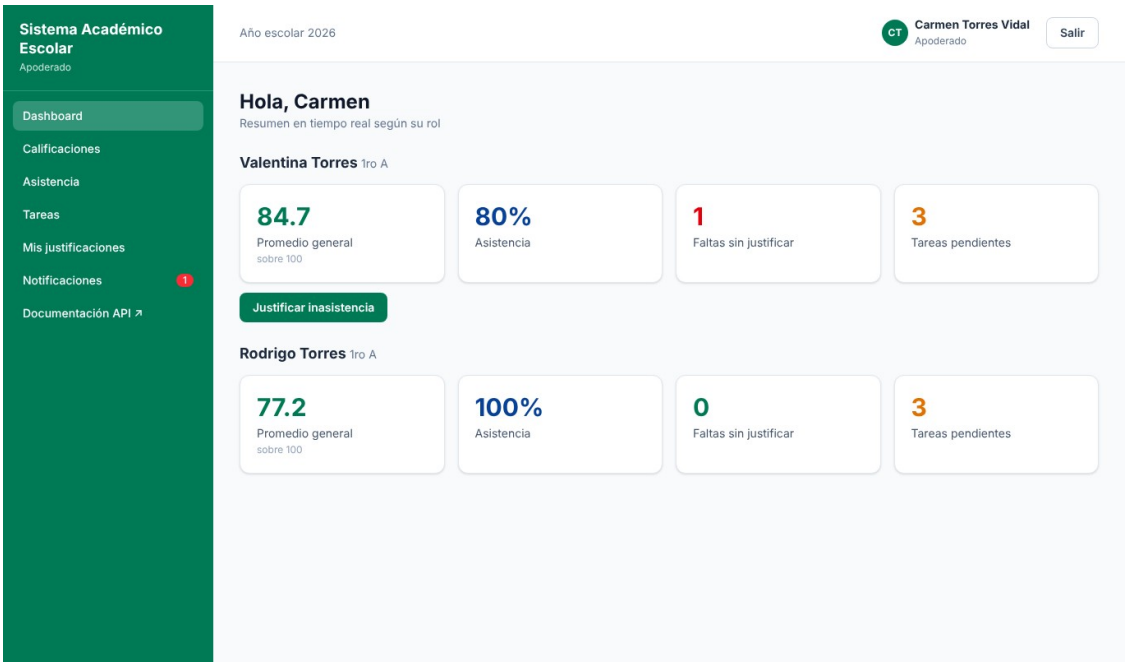


Figura 9. Apoderado: dashboard con el resumen de cada hijo.

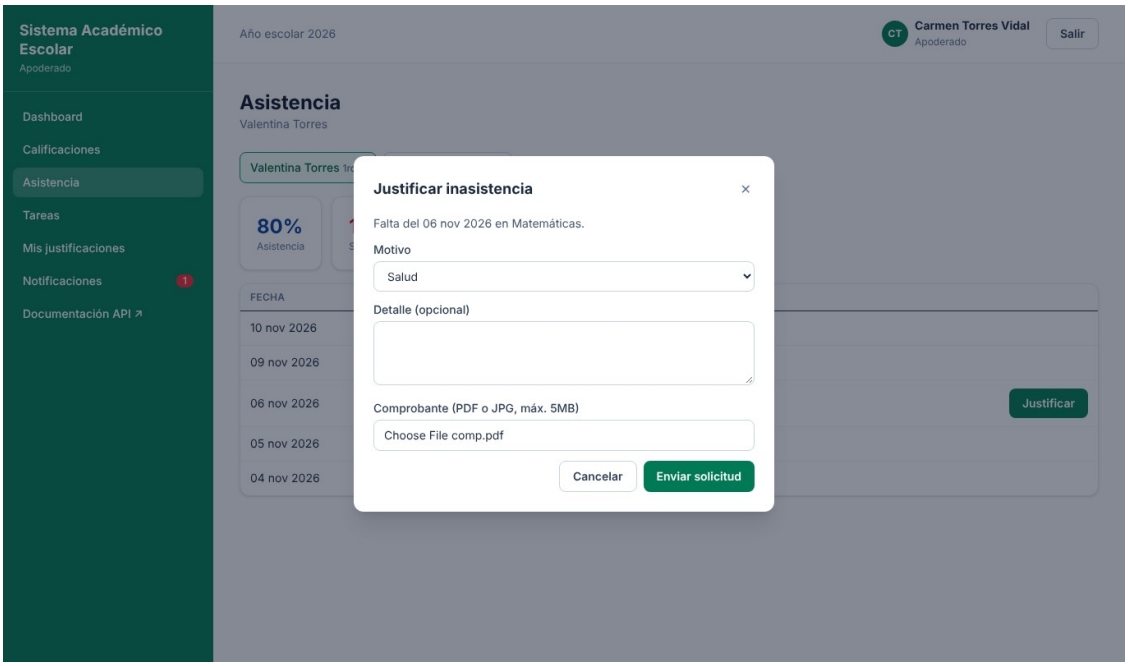


Figura 10. Apoderado: formulario de justificación con comprobante (UC-08).

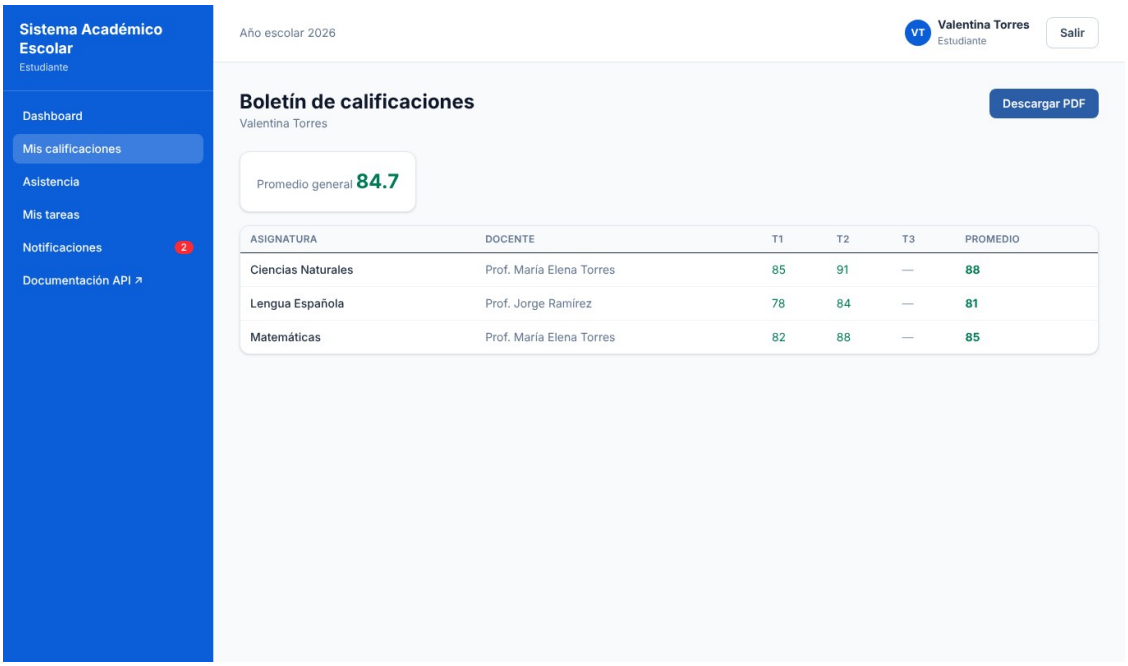


Figura 11. Estudiante: boletín de calificaciones con descarga en PDF (RF-009).

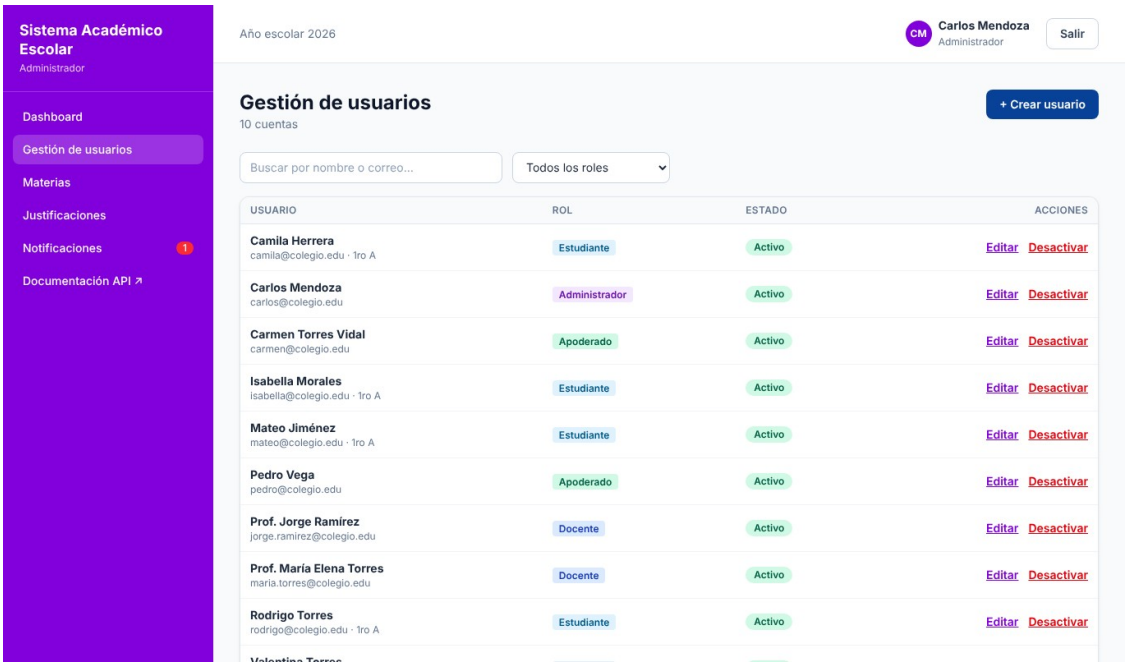


Figura 12. Administrador: gestión de usuarios.



Figura 13. Administrador/Regencia: aprobación de justificantes.



Figura 14. Vista móvil (390 px) del dashboard de apoderado.

7. Trazabilidad con la primera entrega (RF y RNF)

7.1 Requisitos funcionales

Código	Requisito	Implementación	Endpoints / vistas
RF-001	Registro e inicio de sesión con validación OTP	auth.controller, OTP con hash y vigencia 3 min; login con contraseña	`/auth/register`, `/auth/login`, `/auth/otp/*` · Login, Register
RF-002	Roles y permisos diferenciados	authorize + alcance por pertenencia; CRUD de usuarios	`/users`, `/auth/me` · Users
RF-003	Dashboard por rol	dashboard.controller con agregaciones	`/dashboard` · Dashboard
RF-004	Registro y modificación de calificaciones	Upsert masivo `bulkWrite`, rango 0–100, solo docente titular	`/grades/sheet`, `/grades/bulk` · Grades
RF-005	Asistencia diaria por asignatura	Registro por materia y fecha, único por día	`/attendance/sheet`, `/attendance/bulk` · Attendance
RF-006	Alertas por correo ante cambios	notifyUsers: notificación interna + correo SMTP	`/notifications` · Notifications
RF-007	Crear y asignar tareas	CRUD de tareas con fecha límite	`/tasks` · Tasks
RF-008	Envío de archivos adjuntos	multer (5 MB), marca de entrega tardía	`/tasks/:id/submissions` · Tasks
RF-009	Boletín oficial en PDF	pdfkit genera el PDF bajo demanda	`/reports/bulletin/:id` · Grades
RF-010	Solicitud de justificación	Comprobante PDF/JPG, ticket, flujo de aprobación	`/justifications` · Attendance, Justifications

7.2 Requisitos no funcionales

Código	Criterio	Qué se hizo	Estado / verificación
RNF-001	API < 200 ms, página < 2 s	Índices en las consultas frecuentes, `bulkWrite` para cargas masivas, paginación, bundle de ~89 KB gzip.	Diseñado para cumplirlo. Pendiente medir con k6/Lighthouse sobre el despliegue.
RNF-002	JWT/OAuth2, TLS 1.3, AES-256	JWT firmado con expiración y bcrypt implementados. TLS y cifrado en reposo se obtienen en el despliegue (HTTPS del hosting y cifrado de MongoDB Atlas).	JWT ✔️. OAuth2 no implementado. TLS/AES dependen de la infraestructura.
RNF-003	WCAG 2.1 AA	Etiquetas, ARIA, foco visible, salto al contenido, contraste de la paleta, estados con texto.	Aplicado; sin auditoría formal (axe/Lighthouse pendiente).
RNF-004	Responsive y navegadores modernos	Diseño adaptable con Tailwind; verificado en 1366 px y 390 px.	Probado en Chromium. Firefox, Safari y Edge pendientes de verificación manual.
RNF-005	Disponibilidad 99.5%	Endpoint `/api/health`, errores controlados, sin estado en memoria (escala horizontal).	Depende del proveedor de hosting elegido.

8. Pruebas realizadas

Se automatizaron dos niveles de prueba en el backend y se verificó la interfaz en un navegador real:

Nivel	Herramienta	Resultado	Qué cubre
Unitarias (`npm test`)	node:test + supertest	13 / 13 correctas	Salud de la API; 401 sin token, con token falso, manipulado y expirado; registro rechaza roles privilegiados; RBAC; Swagger; 404; validaciones de modelos (nota 0–100, correo, rol, motivo); bcrypt; contenido del JWT.
Integración (`npm run test:integration`)	node:test + supertest + MongoDB real	9 / 9 correctas	Login y OTP completos (código erróneo, correcto y de un solo uso); RBAC por rol; CRUD admin; notas fuera de rango (101 y −1 → 400); docente no titular → 403; upsert sin duplicados; boletín y PDF válido (%PDF); estudiante/apoderado sin acceso a datos ajenos; asistencia → justificación (sin archivo, tipo inválido, 5 MB+, duplicado → 409, apoderado ajeno → 403) → aprobación → falta justificada; tareas con archivo y reenvío; dashboards y notificaciones.
Interfaz (Playwright)	Chromium headless	Sin errores de consola	Recorrido real: login OTP, carga de notas con valor inválido y válido, toma de asistencia, justificación con archivo, boletín, administración y Swagger.

Durante las pruebas se detectó y corrigió que la validación de contraseña como *hook* no se ejecutaba en validaciones sincrónicas; se movió a un validador del propio esquema. También se corrigió un conflicto de orden de estilos que dejaba visible el botón de menú en escritorio.

9. Instalación, ejecución y despliegue

Requisitos: Node.js 18 o superior, npm, Git y MongoDB (local o Atlas). Pasos resumidos (el detalle y la tabla completa de variables están en el `README.md` del repositorio):

Paso	Comandos
1. Clonar	`git clone https://github.com/<usuario>/sistema-academico-escolar.git`
2. Backend	`cd server` · `npm install` · copiar `.env.example` a `.env` y completar `MONGODB_URI` y `JWT_SECRET` · `npm run seed` (opcional) · `npm run dev`
3. Frontend	`cd client` · `npm install` · copiar `.env.example` a `.env` · `npm run dev`
4. Abrir	Aplicación: `http://localhost:5173` · Swagger: `http://localhost:5000/api-docs`

Usuarios de demostración (tras `npm run seed`, contraseña `Demo12345`): administrador `carlos@colegio.edu`, docente `maria.torres@colegio.edu`, estudiante `valentina@colegio.edu` y apoderado `carmen@colegio.edu`.

9.1 Variables de entorno principales

Variable	Uso
`MONGODB_URI`	Cadena de conexión de MongoDB (local o Atlas).
`JWT_SECRET`, `JWT_EXPIRES_IN`	Clave de firma (≥ 32 caracteres en producción) y vigencia del token.
`BCRYPT_SALT_ROUNDS`	Costo del hash (mínimo efectivo 10).
`CLIENT_URL`	Origen permitido por CORS.
`OTP_TTL_MINUTES`, `EXPOSE_DEV_OTP`	Vigencia del OTP; mostrar el código en la respuesta (solo desarrollo).
`SMTP_*`, `MAIL_FROM`	Servidor de correo; vacío = los correos se imprimen en consola.
`VITE_API_URL`, `VITE_API_ORIGIN`	URL de la API y origen del backend para el enlace a Swagger (frontend).

Despliegue sugerido: base de datos en MongoDB Atlas, API en un servicio Node (Render, Railway) con `NODE_ENV=production` y `EXPOSE_DEV_OTP=false`, y frontend estático (Netlify, Vercel) con `VITE_API_URL` apuntando a la API. Si se despliega, agregar aquí los enlaces: Frontend _____ · API _____.

10. Decisiones, limitaciones y trabajo futuro

10.1 Decisiones respecto a la primera entrega

- **Escala de notas 0–100.** El caso de uso UC-03 definió 0–100, mientras el prototipo de Figma mostraba valores sobre 10. Se implementó 0–100 (la escala documentada y validada); el prototipo debería actualizarse para reflejarlo.
- **Almacenamiento de archivos.** El C4 de contexto preveía un servicio Cloud Storage; en esta entrega los archivos se guardan en disco (`server/uploads`), encapsulados en un único módulo para migrar luego a S3 o Cloudinary sin tocar los controladores.
- **Boletín bajo demanda.** El PDF se genera al solicitarlo (no en lote) y refleja las notas vigentes en ese momento.
- **Dos formas de ingreso.** Se mantuvo el OTP por correo del diseño original y se añadió el ingreso con contraseña (bcrypt) que exige esta entrega.

10.2 Limitaciones conocidas

- No se implementó OAuth2 (RNF-002); la autenticación es solo JWT propio.
- Las métricas de rendimiento (RNF-001), la auditoría WCAG y la prueba en Firefox/Safari/Edge quedan pendientes de medir formalmente.
- El token se guarda en `sessionStorage`: simple y se borra al cerrar la pestaña, pero expuesto a XSS; una alternativa más robusta es una cookie `httpOnly` con protección CSRF.
- Las notas se registran por trimestre (T1–T3); no hay ponderación por tipo de evaluación ni cierre formal de periodo (UC-03, A2).

10.3 Trabajo futuro

- Cierre de periodo académico en solo lectura; recuperación de contraseña; refresco de tokens.
- Almacenamiento en la nube, envío de correos en cola y pruebas de carga.
- Aplicación móvil o PWA para apoderados y tablero de indicadores para dirección.

11. Conclusiones

Se implementó un sistema académico completo y coherente con el diseño de la primera entrega: los diez requisitos funcionales están cubiertos de extremo a extremo (base de datos, API protegida y interfaz por rol) y la mayoría de los requisitos no funcionales se atienden en el código; los que dependen de la infraestructura o de mediciones posteriores se señalan de forma explícita.

Los puntos que más valor técnico aportan son la **autorización por pertenencia** (no solo por rol), la **integridad respaldada por índices únicos** en la base de datos, el **flujo de justificación** con ticket y aprobación, y una **suite de pruebas** que valida los flujos críticos contra una base de datos real. El sistema queda listo para su despliegue y como base para las siguientes iteraciones.

Anexo A. Catálogo completo de endpoints

Roles: Admin = administrador; “Autenticado” = cualquier usuario con sesión. Los códigos son los documentados en Swagger. Generado automáticamente desde la especificación OpenAPI.

Método	Ruta	Roles	Códigos	Descripción
GET	`/api/health`	Pública	200	Estado del servicio
POST	`/api/auth/register`	Pública	201, 400, 409	Registro público (estudiante o apoderado) (RF-001)
POST	`/api/auth/login`	Pública	200, 400, 401	Inicio de sesión con correo y contraseña (RF-001)
POST	`/api/auth/otp/request`	Pública	200, 400, 404	Solicitar código OTP por correo (UC-01) (RF-001)
POST	`/api/auth/otp/verify`	Pública	200, 400	Verificar OTP y obtener JWT (UC-01) (RF-001)
GET	`/api/auth/me`	Autenticado	200, 401	Perfil del usuario con sesión activa (RF-002)
GET	`/api/dashboard`	Autenticado	200, 401	Métricas del panel según el rol (RF-003)
GET	`/api/users`	Admin	200, 401, 403	Listar usuarios (paginado, filtros) (RF-002)
POST	`/api/users`	Admin	201, 400, 401, 403, 409	Crear usuario con cualquier rol (RF-002)
GET	`/api/users/stats`	Admin	200, 401, 403	Conteo de usuarios por rol
GET	`/api/users/:id`	Admin	200, 401, 403, 404	Obtener un usuario
PUT	`/api/users/:id`	Admin	200, 400, 401, 403, 404	Actualizar usuario (rol, estado, hijos, contraseña) (RF-002)
DELETE	`/api/users/:id`	Admin	200, 401, 403, 404	Desactivar usuario (baja lógica)
GET	`/api/subjects`	Autenticado	200, 401	Listar materias visibles para el usuario
POST	`/api/subjects`	Admin	201, 400, 401, 403, 409	Crear materia (docente + estudiantes matriculados)
GET	`/api/subjects/:id`	Autenticado	200, 401, 404	Detalle de materia
PUT	`/api/subjects/:id`	Admin	200, 400, 401, 403, 404	Actualizar materia / matrícula
DELETE	`/api/subjects/:id`	Admin	204, 401, 403, 404	Eliminar materia
GET	`/api/grades`	Autenticado	200, 401	Listar calificaciones (acotado por rol) (RF-004)
GET	`/api/grades/sheet`	Docente, Admin	200, 401, 403	Planilla de notas de una materia y periodo (RF-004)
PUT	`/api/grades/bulk`	Docente, Admin	200, 400, 401, 403	Guardar y publicar calificaciones (UC-03) (RF-004)
GET	`/api/grades/bulletin/:studentId`	Autenticado	200, 401, 404	Boletín en JSON de un estudiante (RF-009)
DELETE	`/api/grades/:id`	Docente, Admin	204, 401, 403, 404	Eliminar una calificación
GET	`/api/reports/bulletin/:studentId`	Autenticado	200, 401, 404	Boletín oficial en PDF (RF-009)
GET	`/api/attendance`	Autenticado	200, 401	Listar asistencia (acotada por rol) (RF-005)
GET	`/api/attendance/sheet`	Docente, Admin	200, 401, 403	Lista del día para una materia (RF-005)
POST	`/api/attendance/bulk`	Docente, Admin	200, 400, 401, 403	Registrar asistencia diaria de una materia (RF-005)
GET	`/api/attendance/summary/:studentId`	Autenticado	200, 401, 404	Porcentaje y conteo de asistencia

Método	Ruta	Roles	Códigos	Descripción
GET	`/api/tasks`	Autenticado	200, 401	Listar tareas (con estado de entrega para estudiante/apoderado) (RF-007)
POST	`/api/tasks`	Docente, Admin	201, 400, 401, 403	Crear y asignar una tarea (RF-007)
PUT	`/api/tasks/:id`	Docente, Admin	200, 400, 401, 403, 404	Editar tarea (RF-007)
DELETE	`/api/tasks/:id`	Docente, Admin	204, 401, 403, 404	Eliminar tarea y sus entregas
GET	`/api/tasks/:id/submissions`	Docente, Admin	200, 401, 403, 404	Ver entregas de una tarea (RF-008)
POST	`/api/tasks/:id/submissions`	Estudiante	201, 400, 401, 403, 404	Entregar una tarea con archivo adjunto (máx. 5MB) (RF-008)
GET	`/api/tasks/submissions/:submissionId/file`	Docente, Admin, Estudiante	200, 401, 403, 404	Descargar archivo de una entrega
GET	`/api/justifications`	Apoderado, Admin	200, 401, 403	Listar solicitudes de justificación (RF-010)
POST	`/api/justifications`	Apoderado	201, 400, 401, 403, 409	Solicitar justificación de inasistencia (UC-08) (RF-010)
PATCH	`/api/justifications/:id/review`	Admin	200, 400, 401, 403, 404, 409	Aprobar o rechazar una solicitud (Regencia) (RF-006)
GET	`/api/justifications/:id/file`	Apoderado, Admin	200, 401, 403, 404	Descargar comprobante adjunto
GET	`/api/notifications`	Autenticado	200, 401	Mis notificaciones y total sin leer (RF-006)
PATCH	`/api/notifications/read-all`	Autenticado	200, 401	Marcar todas como leídas
PATCH	`/api/notifications/:id/read`	Autenticado	200, 401, 404	Marcar una notificación como leída